Project Report
Intelligent Temperature Control System

COE332

Wasayf Alfarraj | 441203513

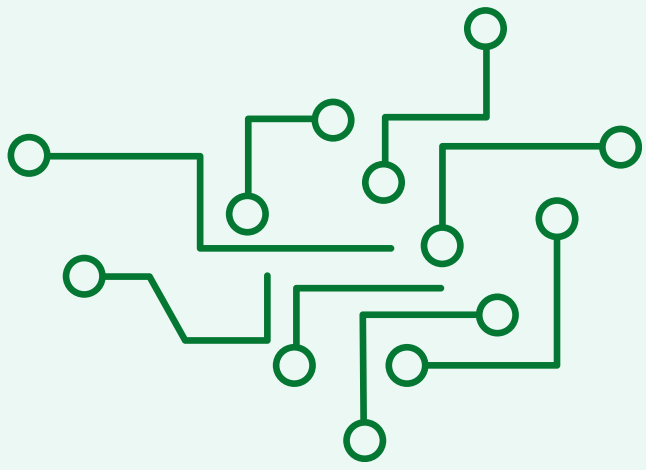
Supervised by: Dr. Wesam

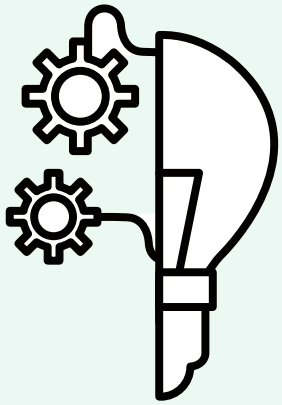
1. Introduction

This project presents the design and implementation of an Intelligent Temperature Control System using the ATmega328P microcontroller. The main objective is to automatically maintain a room's temperature at a desired setpoint of 20°C by controlling a DC fan (for cooling) and a heating element (for warming), based on continuous sensor readings.
The system is important in real-world applications such as server rooms, greenhouses, and smart homes, where maintaining a stable temperature is critical. The project demonstrates how a microcontroller can read analog sensor data, make control decisions using a hysteresis algorithm, and display real-time status on an LCD screen.

Objectives:

- Continuously monitor room temperature using an LM35 analog sensor.
- Compare measured temperature to a target setpoint (20°C).
- Automatically activate the fan or heater based on a hysteresis control algorithm.
- Display current temperature, setpoint, and device states on a 16×2 LCD.





2. Component Description

2.1 ATmega328P Microcontroller

The ATmega328P is an 8-bit AVR RISC-based microcontroller operating at up to 20 MHz. It features 32KB Flash, 2KB SRAM, a 10-bit ADC with up to 8 channels, multiple GPIO ports, and built-in timers. In this project it serves as the central processing unit: it reads the ADC, runs the control logic, drives the actuators, and manages the LCD.

2.2 LM35 Temperature Sensor

The LM35 is a precision analog temperature sensor that outputs a voltage linearly proportional to temperature at a rate of 10 mV per °C. It does not require any external calibration and operates from –55°C to +150°C. Its analog output is connected to ADC Channel 0 (pin PC0) of the microcontroller.

2.3 DC Fan / Motor with Transistor Driver

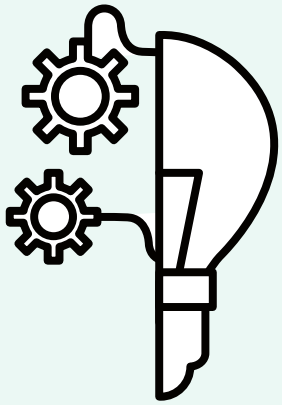
A small DC fan is used as the cooling actuator. Because the ATmega328P GPIO pins can only source ~40 mA, an NPN transistor (e.g., BC547 or 2N2222) is used as a switch. When the control pin (PB0) goes HIGH, the transistor saturates and current flows through the fan motor.

2.4 Heating Element with Transistor

A resistive heating element (or lamp) acts as the warming actuator. It is also driven through an NPN transistor connected to pin PB1. A flyback diode is placed across inductive loads to protect the transistor.

2.5 16×2 LCD Display

The 16×2 character LCD uses a standard HD44780-compatible controller. It is interfaced in 4-bit mode using pins PD0–PD5 of the ATmega328P (RS, EN, D4–D7), reducing the number of required GPIO lines from 10 to 6.



3. Display Interface (16×2 LCD in 4-bit Mode)

The LCD is initialized and operated in 4-bit mode through the following steps:

1. Power-on delay – Wait ≥ 50 ms after VCC rises to 4.5 V.

2. Initialization sequence – Send the nibble 0x03 three times with specific delays, then switch to 4-bit mode by sending 0x02.

3. Function Set – Send 0x28: 4-bit interface, 2 display lines, 5×8 dot font.

4. Display ON – Send 0x0C: display on, cursor off, blink off.

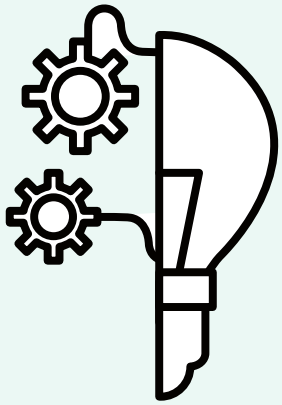
5. Entry Mode – Send 0x06: increment cursor, no display shift.

6. Clear Display – Send 0x01 and wait 2 ms.

Data is sent by splitting each byte into two 4-bit nibbles (high nibble first), placing each on pins D4–D7, and pulsing the Enable (EN) line for each nibble.

Display layout:

Line 1: Temp:23.5C Set:20C
Line 2: Fan:ON Htr:OFF



4. ADC Conversion

The ATmega328P's built-in 10-bit ADC converts the analog voltage from the LM35 into a digital value in the range 0–1023.

Configuration used:

- Reference voltage: AVcc = 5 V (REFS1:0 = 01 in ADMUX)
- ADC clock: 16 MHz ÷ 128 = 125 kHz (ADPS2:1:0 = 111 in ADCSRA)
- Input channel: ADC0 / PC0 (MUX3:0 = 0000)

Conversion Formula:

The ADC output value is related to the input voltage by:

ADC_Value = (V_in / V_ref) × 1023

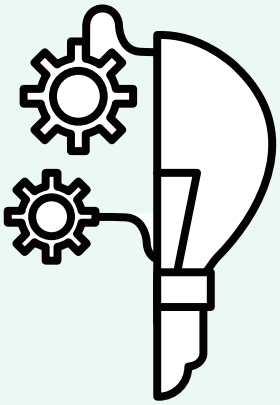
Rearranging to get voltage:

V_in = (ADC_Value / 1023) × V_ref

Since the LM35 outputs 10 mV per °C:

Temperature (°C) = V_in / 0.01 V
= (ADC_Value / 1023) × V_ref / 0.01
= (ADC_Value × 5.0 × 100) / 1023

Example: If ADC reads 460 →Temperature = (460 × 500) / 1023 ≈ 22.5°C



5. Control Algorithm (Hysteresis)

A simple ON/OFF controller with hysteresis is implemented to prevent rapid (chattering)

switching of the actuators when temperature is near the setpoint.

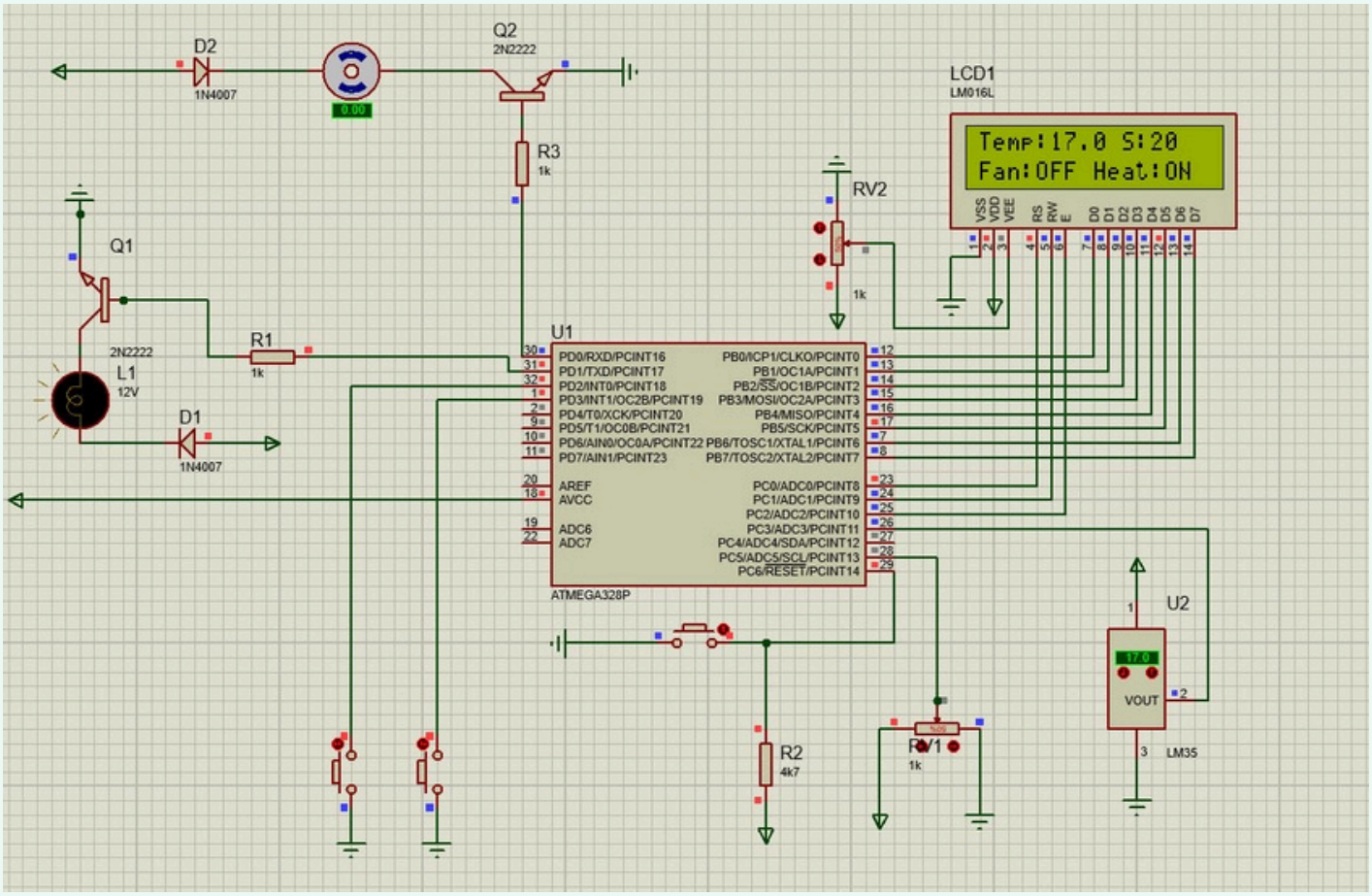
Condition	Fan	Heater
Temperature > 22°C	ON	OFF
Temperature < 18°C	OFF	ON
18°C ≤ Temperature ≤ 22°C	Maintain current state	

The hysteresis band (18°C – 22°C) is ±2°C around the 20°C setpoint.
This avoids the relay/transistor from switching on and off rapidly when temperature hovers near the setpoint.

Simulation Results

Case 1: Temperature Below 18°C

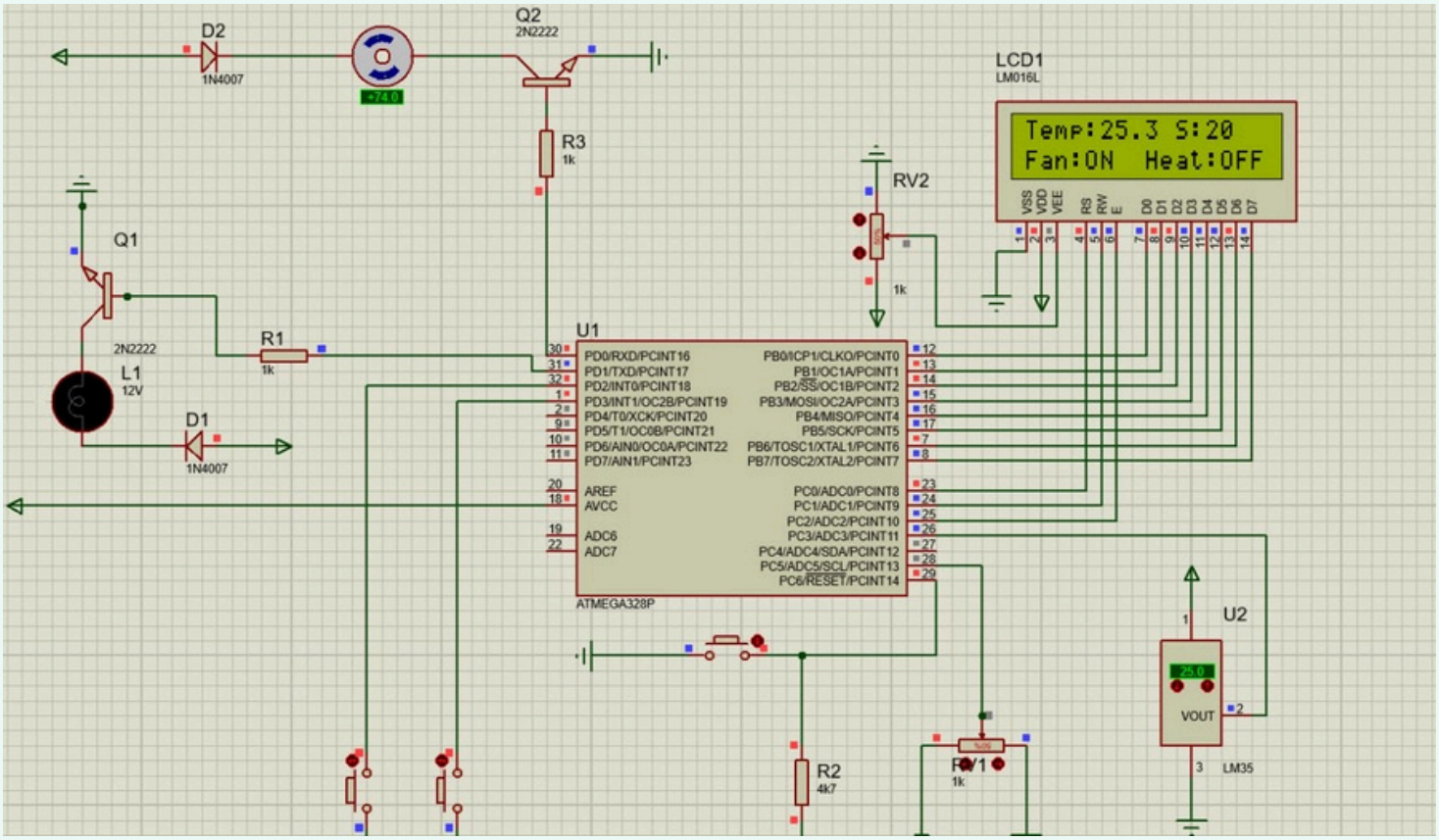
When the measured temperature drops below 18°C, the system interprets this as a cold environment that requires heating. In response, the controller activates the heater to raise the temperature, while the fan remains completely off since cooling is unnecessary. The LCD display reflects this condition by indicating “Heater: ON” and “Fan: OFF”, confirming that the system has entered heating mode.



Simulation Results

Case 2: Temperature Above 22°C

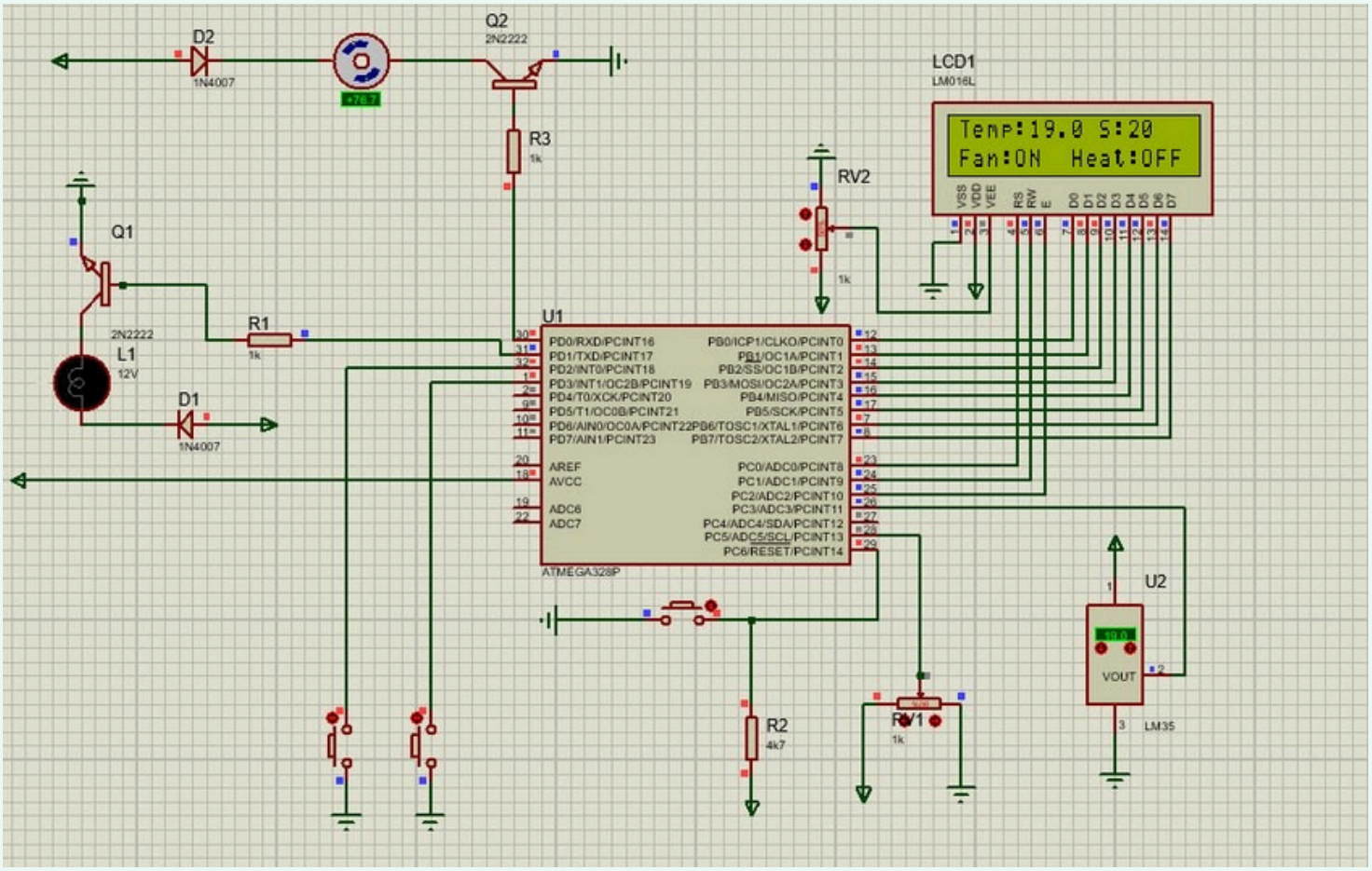
If the temperature rises above 22°C, the system recognizes that the room has become warmer than the desired comfort range. To correct this, the heater is switched off and the fan is turned on to cool the area. The LCD clearly shows “Fan: ON” and “Heater: OFF”, demonstrating that the cooling mechanism has been activated to bring the temperature back down.



Simulation Results

Case 3: Temperature Within 18–22°C (Maintain Previous State)

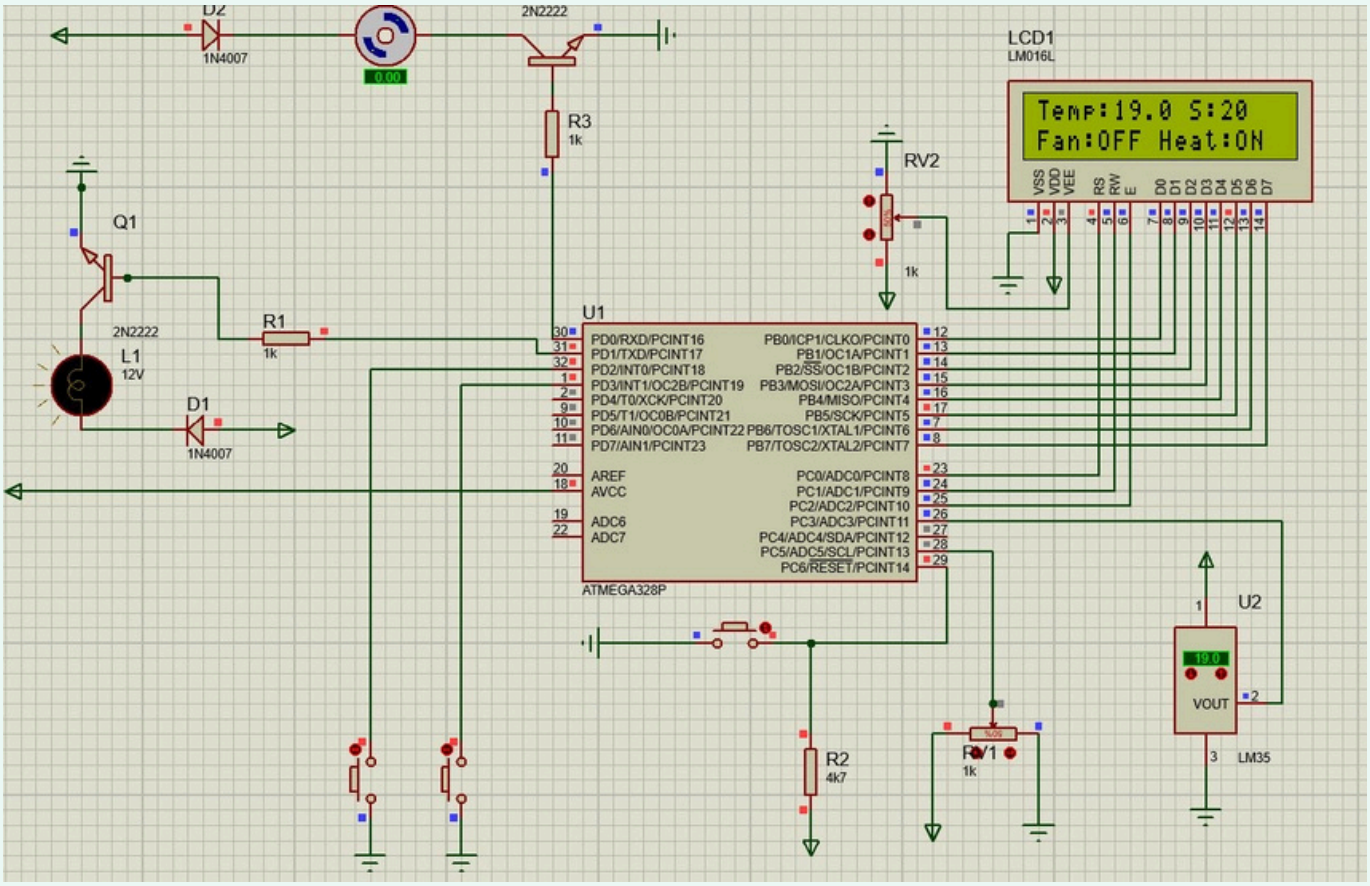
• The temperature has dropped into the hysteresis band (18–22°C) after previously being above 22°C. The system maintains the fan in its previous ON state without activating the heater, demonstrating correct hysteresis behavior

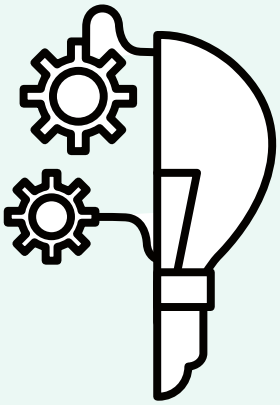


Simulation Results

Case 4: Temperature transitions between states

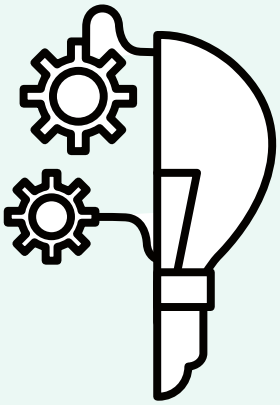
• The temperature is within the hysteresis band (18–22°C) after previously being below 18°C. The system maintains the heater in its previous ON state without activating the fan, further confirming the hysteresis control logic.





Source Code

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>

/* ———— LCD pin mapping ———— */
#define LCD_RS PC0
#define LCD_RW PC1
#define LCD_EN PC2

/* ———— LCD low-level functions ———— */
static void LCD_Command(uint8_t cmd) {
PORTB = cmd;
PORTC &= ~((1 << LCD_RS) | (1 << LCD_RW));
PORTC |= (1 << LCD_EN);
_delay_ms(1);
PORTC &= ~(1 << LCD_EN);
}

static void LCD_Data(uint8_t data) {
PORTB = data;
PORTC |= (1 << LCD_RS);
PORTC &= ~(1 << LCD_RW);
PORTC |= (1 << LCD_EN);
_delay_ms(1);
PORTC &= ~(1 << LCD_EN);
}

static void LCD_Init(void) {
_delay_ms(20);
LCD_Command(0x38);
LCD_Command(0x0C);
LCD_Command(0x06);
LCD_Command(0x01);
_delay_ms(2);
}

static void LCD_String(char *str) {
while (*str) {
LCD_Data(*str++);
}
}

/* ———— ADC configuration ———— */
static void ADC_Init(void) {
ADMUX = (1 << REFS0);
ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1);
}

static uint16_t ADC_Read(uint8_t channel) {
ADMUX = (1 << REFS0) | (channel & 0x07);
ADCSRA |= (1 << ADSC);
while (ADCSRA & (1 << ADSC));
return ADC;
}

/* ———— Main application ———— */
int main(void) {
DDRB = 0xFF;
DDRC = (1 << LCD_RS) | (1 << LCD_RW) | (1 << LCD_EN);
DDRD |= (1 << PD0) | (1 << PD1);
PORTD |= (1 << PD2) | (1 << PD3);
LCD_Init();
ADC_Init();

int setpoint   = 20;
const int SET_MIN = 10;
const int SET_MAX = 35;
char lcd_buf[16];
```



Source Code

```
while (1) {
 /* ------- Temperature acquisition ------- */
 uint16_t adc_value = ADC_Read(3);
 float temp_c = (adc_value * 500.0) / 1024.0;

 int t_whole = (int)temp_c;
 int t_dec = (int)((temp_c - t_whole) * 10);

 /* ------- Control logic (18-22 °C hysteresis) ------- */
 if (temp_c > 22.0) {
 PORTD |= (1 << PD0); // Fan ON
 PORTD &= ~(1 << PD1); // Heater OFF
 }
 else if (temp_c < 18.0) {
 PORTD &= ~(1 << PD0); // Fan OFF
 PORTD |= (1 << PD1); // Heater ON
 }
 // 18 <= temp <= 22 : maintain current state (no else)

 /* -------- Button handling (UP / DOWN with debounce) -------- */
 uint8_t buttons_state = PIND & ((1 << PD2) | (1 << PD3));

 if (buttons_state != ((1 << PD2) | (1 << PD3))) {
 _delay_ms(40);
 buttons_state = PIND & ((1 << PD2) | (1 << PD3));

 if (!(buttons_state & (1 << PD2))) {
 if (setpoint < SET_MAX) {
 setpoint++;
 }
 }
 if (!(buttons_state & (1 << PD3))) {
 if (setpoint > SET_MIN) {
 setpoint--;
 }
 }

 while ((PIND & ((1 << PD2) | (1 << PD3))) != ((1 << PD2) | (1 << PD3))) {
 // blocking wait
 }
 }

 /* -------- LCD refresh -------- */
 LCD_Command(0x01);
 _delay_ms(2);

 // first line: temperature + setpoint
 LCD_Command(0x80);
 sprintf(lcd_buf, "Temp:%d.%d S:%d", t_whole, t_dec, setpoint);
 LCD_String(lcd_buf);

 // second line: fan / heater states
 LCD_Command(0xC0);
 sprintf(lcd_buf,
 "Fan:%s Heat:%s",
 (PORTD & (1 << PD0)) ? "ON " : "OFF",
 (PORTD & (1 << PD1)) ? "ON " : "OFF");
 LCD_String(lcd_buf);

 _delay_ms(100);
}

}
```

Click here to see the video

Conclusion

This project successfully demonstrated the design and implementation of an Intelligent Temperature Control System using the ATmega328P microcontroller. The key achievements are:

• **Accurate Temperature Sensing**: The LM35 sensor and 10-bit ADC combination provided reliable temperature readings with 0.1°C display resolution.

• **Effective Hysteresis Control**: The ±2°C hysteresis band prevented rapid switching of the fan and heater, extending actuator life and improving system stability.

• **Real-time Display**: The 16×2 LCD successfully displayed the current temperature, setpoint, and the ON/OFF state of both actuators at all times.

• **Modular Code Structure**: The firmware was organized into separate, well-commented functions for ADC reading, LCD control, and actuator management, making it easy to maintain and extend.

Future improvements could include a PID control algorithm for smoother temperature regulation, user-adjustable setpoint via push buttons, and data logging over UART.